IV.5.3. Memoria virtuală

Ideea de pornire

Problema

- aplicaţiile - consum mare de memorie
- memoria disponibilă - insuficientă

Cum se poate rezolva?

- capacitatea discului hard - foarte mare
- nu toate zonele de memorie ocupate sunt accesate la un moment dat

Memoria virtuală

Soluţia - memoria virtuală (*swap*)

- unele zone de memorie - evacuate pe disc
- când este nevoie de ele, sunt aduse înapoi în memorie

Cine gestionează memoria virtuală?

- sunt necesare informaţii globale
- sistemul de operare

Fişierul de paginare

- conţine zonele de memorie evacuate pe disc
- informaţii pentru regăsirea unei zone stocate
  - adresele din memorie
  - programul căruia îi aparţine
  - dimensiunea
  - etc.

Politica de înlocuire (1)

- problema - aceeaşi ca la memoria cache
- aducerea unei zone de memorie din fişierul de paginare implică evacuarea alteia
  - care?
- scop - minimizarea acceselor la disc
- politică ineficientă → număr mare de accese la disc → scăderea vitezei

Politica de înlocuire (2)

- set de lucru (*working set*) - zonele de memorie necesare programului la un moment dat
- uzual mult mai mic decât totalitatea zonelor folosite de program
- dacă încape în memorie - puţine accese la disc

Politica de înlocuire (3)

- se va selecta pentru evacuare zona care nu va fi necesară în viitorul apropiat
- nu se poate şti cu certitudine - estimare
  - pe baza comportării în trecutul apropiat
- paginare la cerere (*demand paging*) - evacuare pe disc numai dacă este strict necesar

Implementare

- prin intermediul mecanismelor de gestiune a memoriei, deja discutate
  - dacă un program încearcă să acceseze o locație aflată temporar pe disc, este necesar același tip de detecție
  - memoria virtuală poate fi folosită împreună atât cu segmentarea, cât şi cu paginarea
- rolul sistemului de întreruperi - sporit

Accesul la memorie (1)

Cazul paginării

1. programul precizează adresa virtuală
2. se determină pagina din care face parte
3. se caută pagina în tabelul de paginare
4. dacă pagina este găsită - salt la pasul 9
5. generare excepţie
6. rutina de tratare caută pagina în fişierul de paginare

Accesul la memorie (2)

Cazul paginării (cont.)

7. dacă pagina nu este în fişierul de paginare - programul este terminat
8. se aduce pagina în memoria fizică
9. se determină cadrul de pagină corespunzător
10. calcul adresă fizică
11. acces la adresa calculată

Reducerea acceselor la disc (1)

- duce la creşterea performanţei
- o pagină este salvată pe disc şi readusă în memorie de mai multe ori
- readucerea în memorie - copia de pe disc nu este ştearsă
- pagina şi copia sa de pe disc sunt identice până la modificarea paginii din memorie

Reducerea acceselor la disc (2)

- evacuarea unei pagini din memorie
  - dacă nu a fost modificată de când se află în memorie - nu mai trebuie salvată
  - util mai ales pentru paginile de cod
- este necesar sprijin hardware pentru detectarea acestei situaţii
  - este suficient să fie detectate operaţiile de scriere

Reducerea acceselor la disc (3)

- tabelul de paginare - structură extinsă
  - fiecare pagină are un bit suplimentar (*dirty bit*)
  - indică dacă pagina a fost modificată de când a fost adusă în memorie
  - resetat la aducerea paginii în memorie
- instrucţiune de scriere în memorie
  - procesorul setează bitul paginii care conţine locaţia modificată

IV.5.4. Comunicarea între procese

Comunicare (1)

- pentru a putea coopera, procesele trebuie să-și poată transmite date
  - uneori volume mari
- implementare fizică - zone de memorie comune
  - variabile partajate
  - structuri de date mai complexe, prevăzute cu metode specifice de acces

Comunicare (2)

- este necesar ca două sau mai multe procese să acceseze aceeași zonă de memorie
  - același segment să apară simultan în tabelele de descriptori ale mai multor procese
  - același cadru de pagină să apară simultan în tabelele de paginare ale mai multor procese
- în oricare caz, sistemul de operare controlează zonele comune
  - iar procesele sunt conștiente de caracterul partajat al acestora

Excludere mutuală (1)

- accesul la o resursă comună poate dura
  - și poate consta în mai multe operaţii
- apare pericolul interferenţelor
- exemplu
  - un proces începe accesul la o variabilă comună
  - înainte de a termina, alt proces începe să o acceseze
  - variabila poate fi modificată în mod incorect

Excludere mutuală (2)

- accesul la o resursă comună - doar în anumite condiţii
- excludere mutuală
  - la un moment dat, un singur proces poate accesa o anumită resursă
- mecanisme de control
  - semafor - cel mai simplu
  - structuri partajate ale căror metode de acces asigură excluderea mutuală (ex. monitor)

Implementare

- accesul la o resursă poate fi controlat (și blocat) doar de către sistemul de operare
- deci orice formă de accesare a unei resurse partajate implică un apel sistem
- dacă se lucrează la nivel jos (variabile partajate + semafoare), este sarcina programatorului să se asigure că apelul este realizat corect

Fire de execuție - comunicare

- în cazul firelor de execuție ale aceluiași proces, variabilele globale sunt automat partajate
  - viteză mai mare
  - crește riscul erorilor de programare
- necesitatea excluderii mutuale este prezentă și aici

IV.5.5. Utilizarea MMU

Hardware

Cazul Intel

- segmentarea
  - nu poate fi dezactivată
  - dar poate fi "evitată" prin software
- paginarea
  - poate fi activată/dezactivată

Sistemul de operare

Cazurile Windows, Linux

- segmentarea
  - nu este utilizată în practică
  - toate segmentele sunt dimensionate asfel încât să acopere singure întreaga memorie
- paginarea
  - pagini de 4 KB
  - Windows poate folosi şi pagini de 4 MB

Utilitatea MMU (1)

Avantaje

- protecţie la erori
- o aplicaţie nu poate perturba funcţionarea alteia
- verificările se fac în hardware
  - mecanism sigur
  - viteză mai mare

Utilitatea MMU (2)

Dezavantaje

- gestiune complicată
- memorie ocupată cu structurile de date proprii
  - tabelul de descriptori
  - tabelul de paginare
- viteză redusă - dublează numărul acceselor la memorie (sau mai mult)

Utilitatea MMU (3)

Concluzii

- scăderea de performanţă poate fi compensată folosind cache-uri
- procesoarele de azi oferă suficientă viteză
- sisteme multitasking - risc mare de interferenţe
- mecanismele MMU trebuie folosite